



Registries

Registries are (ordered) key/value maps. They are the main web client extension points: many features provided by the Odoo javascript framework simply look up into a registry whenever it needs a definition for some object (such as fields, views, client actions or services). Customizing the web client is then simply done by adding specific values in the correct registry.

```
import { Registry } from "@web/core/registry";

const myRegistry = new Registry();

myRegistry.add("hello", "odoo");

console.log(myRegistry.get("hello"));
```

A useful feature of registries is that they maintain a set of sub registries, obtained by the `category` method. If the sub registry does not exist yet, it is created on the fly. All registries used by the web client are obtained in such a way from one root registry, exported in `@web/core/registry`.

```
import { registry } from "@web/core/registry";

const fieldRegistry = registry.category("fields");
const serviceRegistry = registry.category("services");
const viewRegistry = registry.category("views");
```

Registry API

`class Registry()`

Creates a new registry. Note that a registry is an event bus, so one can listen to the `UPDATE` event if necessary. Registries are ordered: the `getAll` method returns a list of values ordered according to their sequence number.

`Registry.add (key, value[, options])`

value (*any()*) – value for the new entry

options (*Object()*) – options

[options.force] (*boolean()*) – do not throw if key already exists

[options.sequence] (*number()*) – sequence number (useful to order entries)

Returns:

Registry

Inserts a value at a specific key. If the key is already used, this method throws an error (unless the option `force` is set to true). The option `sequence` is useful to insert the value at a specific position. This method also triggers an `UPDATE` event.

Returns the same registry, so `add` method calls can be chained.

Registry.get (key[, defaultValue])

Arguments:

key (*string()*) – key for the entry

any (*defaultValue()*) – return value if no entry for key exists

Returns the value corresponding to the `key` argument. If the registry does not contain that key, this method returns `defaultValue` if given, or throws an error otherwise.

Registry.contains (key)

Arguments:

key (*string()*) – key for the entry

Returns:

boolean

Returns `true` if `key` is present in the registry

Registry.getAll ()

Returns:

any[]

Returns the list of all elements in the registry. It is ordered according to the sequence numbers.

Registry.remove (key)

Removes a key/value pair from the registry. This operation triggers an `UPDATE` event.

Registry.category (subcategory)

Arguments:

subcategory (`string()`) – the name for the sub category

Returns:

Registry

Returns the sub registry associated with the `subcategory` . If it does not exist yet, the sub registry is created on the fly.

Reference List

Category	Content
effects	implementation for all available effects
formatters	utility functions to format values (mostly used for field values)
main_components	top level components
parsers	utility functions to parse values (mostly used for field values)
services	all services that should be activated
systray	components displayed in the systray zone in the navbar
user_menuitems	menu items displayed in the user menu (top right of navbar)

Effect registry

The `effects` registry contains the implementations of all available effects. See the section on the [effect service](#) for more details.

Formatter registry

The `formatters` registry contains functions to format values. Each formatter has the following API:

format(value[, options])

Returns:

string

Formats a value and returns a string

See also

- [Parsers registry](#)

Main components registry

The main component registry (`main_components`) is useful for adding top level components in the web client. The webclient has a `MainComponentsContainer` as direct child. This component is basically a live representation of the ordered list of components registered in the main components registry.

API

```
interface {
    Component: Owl Component class
    props?: any
}
```

For example, the `LoadingIndicator` component can be added in the registry like this:

```
registry.category("main_components").add("LoadingIndicator", {
    Component: LoadingIndicator,
});
```

Parser registry

The `parsers` registry contains functions to parse values. Each parser has the following API:

`parse(value[, options])`

Arguments:

value (`string()`) – a string representing a value

options (`Object()`) – various options (parser specific)

Returns:

can fail and throw errors.

See also

- [Formatters registry](#)

Service registry

The service registry (category: `services`) contains all [services](#) that should be activated by the Odoo framework.

```
import { registry } from "@web/core/registry";

const myService = {
  dependencies: [...],
  start(env, deps) {
    // some code here
  }
};

registry.category("services").add("myService", myService);
```

Systray registry

The systray is the zone on the right of the navbar that contains various small components, that usually display some sort of information (like the number of unread messages), notifications and/or let the user interact with them.

The `systray` registry contains a description of these systray items, as objects with the following three keys:

- `Component` : the component class that represents the item. Its root element should be a `<li>` tag, otherwise it might not be styled properly.
- `props (optional)` : props that should be given to the component
- `isDisplayed (optional)` : a function that takes the [env](#) and returns a boolean. If true, the systray item is displayed. Otherwise it is removed.

For example:

```
// some component ...  
  
}  
  
registry.category("systray").add("myAddon.myItem", {  
    Component: MySystrayItem,  
});
```

The systray registry is an ordered registry (with the `sequence` number):

```
const item = {  
    Component: MySystrayItem  
};  
registry.category("systray").add("myaddon.some_description", item, { sequence: 43 })
```

The sequence number defaults to 50. If given, this number will be used to order the items. The lowest sequence is on the right and the highest sequence is on the left in the systray menu.

Usermenu registry

The user menu registry (category: `user_menuitems`) contains all menu items that are shown when opening the user menu (the navbar element with the user name, on the top right).

User menu items are defined by a function taking the `env` and returning a plain object, containing the following information:

- `description` : the menu item text,
- `href` : (optional) if given (and truthy), the item text is put in a `a` tag with given attribute `href`,
- `callback` : callback to call when the item is selected,
- `hide` : (optional) indicates if the item should be hidden (default: `false`),
- `sequence` : (optional) determines the rank of the item among the other dropdown items (default: 100).

The user menu calls all the functions defining items every time it is opened.

Example:

```
return {  
    description: env._t("Technical Settings"),  
    callback: () => { env.services.action_manager.doAction(3); },  
    hide: (Math.random() < 0.5),  
};  
});
```

Get Help

[Contact Support](#)[Ask the Odoo Community](#)